C2 High Performance Computing Coursework Report

1. Introduction

This report documents the incremental optimisation of an in-place Cholesky factorisation routine for symmetric positive-definite matrices, stored as a 1D flattened array in row-major order. The optimisation process is developed through a series of single-threaded optimisations (v0.0-v0.4) to multi-threaded optimisations (v1.1-v1.2). All optimised versions are benchmarked on the CSD3 Ice Lake partition, compiled with Intel icpx compiler using -O3 -xHost compiler flag, unless stated otherwise, which enables additional compiler and hardware targeted optimisations.

All optimisation performances are benchmarked using omp_get_wtime() for measuring wall clock time, with timer precision 1e-6s. Each configuration was repeatedly measured 5 times, with the mean and standard deviation (Bessel-corrected estimate) of the wall clock time being reported. Algorithm correctness is tested and confirmed, before every benchmark run, via an integrated test suite through the Slurm script. Any newly updated algorithms that did not pass the test suite will be determinate and error will be reported, before the benchmark run begins.

This set of optimisations are algorithmic improvements based on the given base case solution, which has a complexity of $O(n^3)$. The project does not aim to alter the mathematical foundation of the solution.

2. Baseline implementation (v0.0)

2.1 Baseline routine design

The baseline routine is implemented based on the coursework specification, with several deliberate modification for robustness and correctness:

- **Loop index type**: Changed from 'int' as suggested in the coursework specification to 'long', to support matrix dimensions up to N=100,000 without integer overflow.
- **Input validation checks**: 5 input validation checks are implemented within the routine: Checking for successful memory allocation and initialisation of input matrix, valid input matrix dimension (N>0; N<=100,000), valid input matrix property i.e. must be symmetric; valid matrix property i.e., must be positive-definite. The first 4 checks are excluded in the performance benchmarking region, while the positive-definiteness check (diagonal element > very small positive value) is performed at every diagonal step within the timed, main factorisation loop. This marginally penalises timing but ensures the routine do not silently produce incorrect output due to invalid input matrix.
- **__restrict__ qualifier**: This qualifier is applied to the input pointer, for informing compiler that no aliasing will occur, enabling more aggressive optimisation.

2.2 Compiler flags and partition comparison

The baseline was benchmarked with both -O0 (no optimisation) and -O3 -xHost (processor-specific optimisation), on both the Cascade Lake and Ice Lake partitions.

N	-O0 Cascade Avg (s)	-O0 Cascade Std (s)	-O0 Icelake Avg (s)	-O0 Icelake Std (s)	-O3 -xHost Cascade Avg (s)	-O3 -xHost Cascade Std (s)	-O3 -xHost Icelake Avg (s)	-O3 -xHost Icelake Std (s)
100	0.001349	0.000018	0.000909	0.000011	0.000281	0.000007	0.000187	0.000004
500	0.241487	0.002526	0.148377	0.000217	0.068460	0.000121	0.038122	0.000189
1000	1.789275	0.027989	1.204476	0.008305	0.624417	0.001260	0.425013	0.000636
2000	18.820313	0.039889	12.432337	0.088256	6.697941	0.024784	5.468103	0.028597

Table 1: Baseline implementation (v0.0) average time and standard deviation (s) comparison across compiler flags and partitions, benchmarked with various matrix dimensions. Cascade is used shorthand for Cascade Lake.

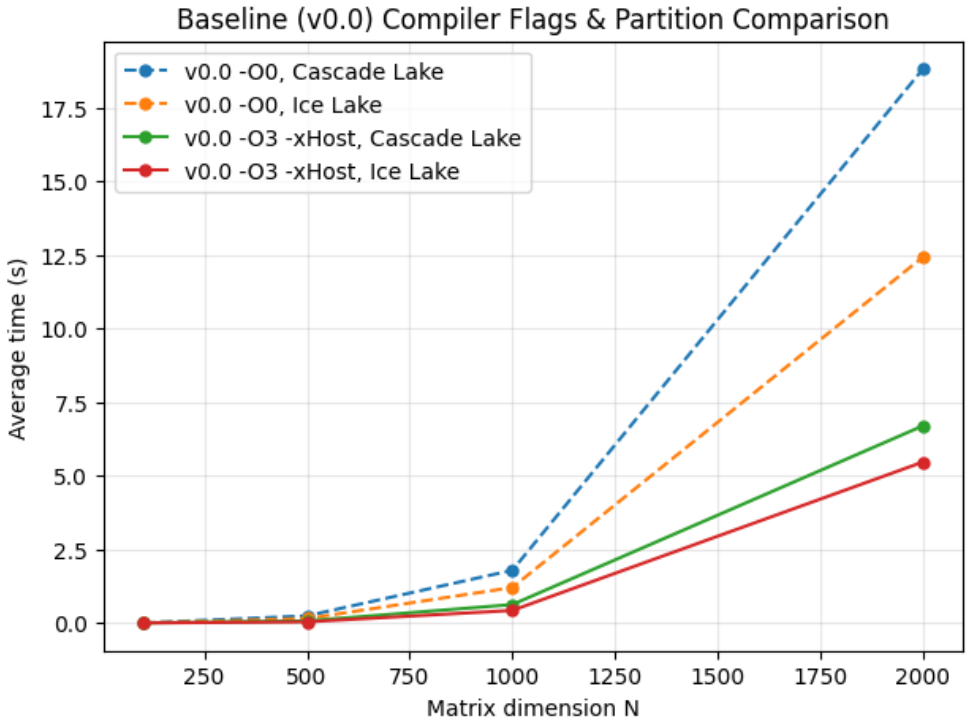


Figure 1: Baseline implementation (v0.0) average execution time comparison across compiler flags and partitions, benchmarked with various matrix dimension.

The compiler optimisation flag speeds up the performance at all matrix sizes on both partitions. The Ice Lake partition is consistently faster than Cascade Lake. And hence all subsequent benchmarks use -O3 -xHost compiler flag on Ice Lake partition. The Intel icpx compiler was chosen since CSD3 has Intel hardware, investigating alternative compilers (e.g., g++) was considered but deprioritised given the limited timeframe, as algorithmic and parallelisation improvements will be expected to achieve a greater performance gain.

3. Single-threaded optimisation (v0.1-v0.4)

All single-threaded optimisations are compiled with -O3 -xHost compiler flag on Ice Lake partition. OpenMP environment variable is needed due to the use of timer, so set to OMP_NUM_THREADS=1.

3.1 v0.1: Loop re-ordering (avoid strided memory access) and hoisting

- **Loop reordering for avoiding strided access**: The submatrix update was re-ordered from j-outer/i-inter to i-outer/j-inter loop structure, so that contiguous read and write can both be achieved for **c[n*i+j]** and **c[n*p+j]**.
- **Hoisting**: The **c[n*i+p]** is calculated and stored in a local variable before the innermost j-loop, eliminating the redundancy for repeated calculation for a fixed value with regard to change in j.

Great optimisation performance is observed, as expected. For example, at N=2000, average execution time decreased from 5.47s to 1.44s from the previous best version, achieving ~3.8 times speedup.

3.2 v0.2: Reciprocal multiplication

- **Reciprocal multiplication instead of division**: Division by the diagonal element was replaced by the less expensive multiplication of the pre-computed inverse. Floating point division is typically slower.

Moderate optimisation performance is observed, e.g., at N=5000, average execution time decreased from 35.85s to 34.84s from the previous version. This might be because the compiler has already, at least partially, optimised this due to the use of compiler flag. And because the submatrix update dominates the execution time. A marginal increase in execution time is observed at smaller matrix dimensions (n<=1000), likely attributed to the measurement variation.

3.3 v0.3: Symmetry

- **Upper-triangle update due to symmetry**: Since the Cholesky factorisation algorithm is implemented in a symmetrical fashion and the output matrix specified is also symmetric, this property can be leveraged in optimisation to roughly half of the main operations. Firstly, the row update is kept while column update is removed, since only one side needs to be updated due to symmetry. Row update is chosen over column updates for contiguous access. Secondly, for the submatrix update, only the upper triangle is updated due to symmetry.
- **Final copy**: Additional operations are needed to copy the upper triangle elements to the lower triangle. Here in implementation, I used j-outer/i-inner loop structure to prioritise contiguous writes over contiguous read, since strided write is more costly due to cache-coherence.

Despite the additional copy operations, this optimisation is still beneficial since the dominating submatrix update is almost halved. As evidenced in the execution time records, at N=5000, the execution time decreased from 34.84s to 16.39s with ~2.1 times speedup.

3.4 v0.4: SIMD and branch prediction

- #**pragma omp simd**: Data-level parallelisation is achieved through using OpenMP Pragma, applied to all 3 innermost loops in the routine. Single instruction multiple data (SIMD) vectorisation is achieved through leveraging the multiple execution units in the processor, to operate on multiple data elements per instruction.
- **__builtin_expect**: The positive-definiteness check needs to be positioned inside the benchmarked section for ensuring valid input and avoid the routine to silently outputting incorrect output. This __builtin_expect marks the positive-definiteness error path as unlikely, and hence help with branch prediction and improve instruction level parallelisation.

This optimisation version showed a moderate-to-good improvement in execution time. This might be attributed to compiler is already vectorising some loops with the compiler flag.

3.5 Summary of the single-threaded optimisation performance

n	v0.0 Avg (s)	v0.0 Std (s)	v0.1 Avg (s)	v0.1 Std (s)	v0.2 Avg (s)	v0.2 Std (s)	v0.3 Avg (s)	v0.3 Std (s)	v0.4 Avg (s)	v0.4 Std (s)
100	0.000187	0.000004	0.000121	0.000002	0.000135	0.000003	0.000071	0.000002	0.000039	0.000002
500	0.038122	0.000189	0.017316	0.000185	0.018777	0.000100	0.009527	0.000130	0.004924	0.000027
1000	0.425013	0.000636	0.147296	0.000227	0.159000	0.000406	0.085346	0.000331	0.056283	0.000287
2000	5.468103	0.028597	1.436534	0.039501	1.329503	0.005937	0.663676	0.001100	0.450845	0.001358
3000	-	-	6.523808	0.079054	6.133880	0.055720	2.579256	0.075275	1.618427	0.036542
4000	-	-	17.368254	0.056339	16.824307	0.069455	7.421129	0.009192	5.650543	0.149379
5000	-	-	35.845882	0.553149	34.840939	0.063529	16.386453	0.200771	13.164208	0.204666
8000	-	-	-	-	-	-	76.491817	0.067520	61.548106	0.388576
10000	-	-	-	-	-	-	-	-	122.821552	0.548808
15000	-	-	-	-	-	-	-	-	414.014815	1.533023

Table 2: Single-threaded optimisation implementations (v0.0-v0.4) average time and standard deviation (s) comparison (Ice Lake partition, -O3 -xHost compiler flag, 1 thread), benchmarked with various matrix dimensions. Lower triangle elements were not benchmarked due to the limited time constraints.

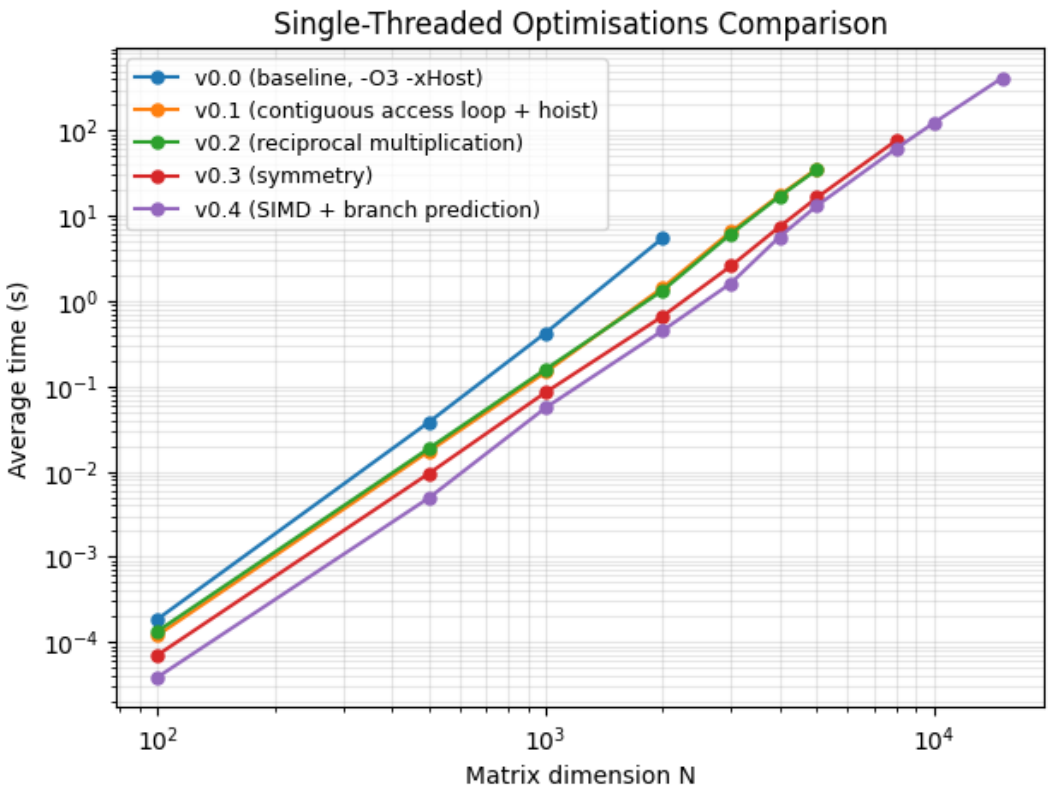


Figure 2: Average execution time (s) against matrix dimension, for all single-threaded optimisation implementations and baseline implementation (v0.0-v0.4) (Ice Lake partition, -O3 -xHost compiler flag, 1 thread). Both axes in log-scale for visualisation.

All single threaded optimisations showed improved execution time at sufficiently large matrix dimensions. The most effective optimisation techniques are avoiding strided access with loop reordering and hoisting, and leveraging the symmetry nature of the algorithm. Some relatively modest performance gain suggests compiler flags already enable optimisation to some extent. The -xHost flag generate architecture specific instructions and should be adapted if porting to a different platform

4. Multi-threaded optimisations with OpenMP (v1.1)

All performance results are obtained via compilation with -O3 -xHost compiler flag on Ice Lake partition.

4.1 v1.1: OpenMP implementations

OpenMP directives and clauses are implemented at the following places:

- **#pragma omp parallel**: Wrapped the entire factorisation for loops from the p-loop.
- **#pragma omp single**: Ensure only one thread will execute the diagonal element update. Implicit barrier already exists, and hence explicit barrier is not implemented.

- **#pragma omp for simd schedule(static)**: Multi-thread optimisation for the row updates. Simd pragma is carried over from the previous version. Static scheduler is used as workload per iteration is the same.
- **#pragma omp for schedule(runtime)**: Multi-thread optimisation for the submatrix (upper triangle elements only) updates. Since workload varies, scheduling is set in the environment variable and is automatically tested (via Slurm script settings) across a range of schedule types and chunk sizes, as opposed a simple choice of static scheduler.
- **#pragma omp for schedule(runtime)**: Multi-thread optimisation for the final copy updates. Again, since workload varies, environment variable is used and tuned.
- **#pragma omp barrier**: Used at the end of row updates, submatrix updates, and final copy updates, to ensure explicit barriers for all threads, although implicit barriers may already in place.
- The other 2 simd pragma implemented previously remain unchanged.

4.2 Scheduling and Thread Count Tuning

Four schedule configurations were tested across 5 thread counts at N=10,000 (Table 3). A large matrix dimension was chosen since multi-threaded parallelisation is most valuable where computational work outweighs thread management overhead. On CSD3 Ice Lake nodes, hyperthreading is disabled, 76 threads represent the maximum available parallelism. A more extensive tuning was considered but didn't implemented due to limited time. The tuning was handled automatically by the slurm_icelake_test.sh script.

Threads	static,32 Avg (s)	static,32 Std (s)	dynamic,16 Avg (s)	dynamic,16 Std (s)	dynamic,32 Avg (s)	dynamic,32 Std (s)	guided,16 Avg (s)	guided,16 Std (s)
1	111.61	15.08	111.65	15.19	111.84	15.53	111.12	14.08
8	19.44	1.52	17.04	0.83	17.16	0.71	21.70	0.48
32	12.58	0.39	12.33	0.30	12.48	0.44	16.32	0.66
56	11.07	0.27	11.40	0.09	11.61	0.27	15.00	0.08
76	9.88	0.33	10.90	0.03	11.18	0.03	14.50	0.03

Table 3: v1.1 scheduling tuning at N=10,000, Ice Lake partition (76 threads available per node), -O3 -xHost compiler flag across different number of threads

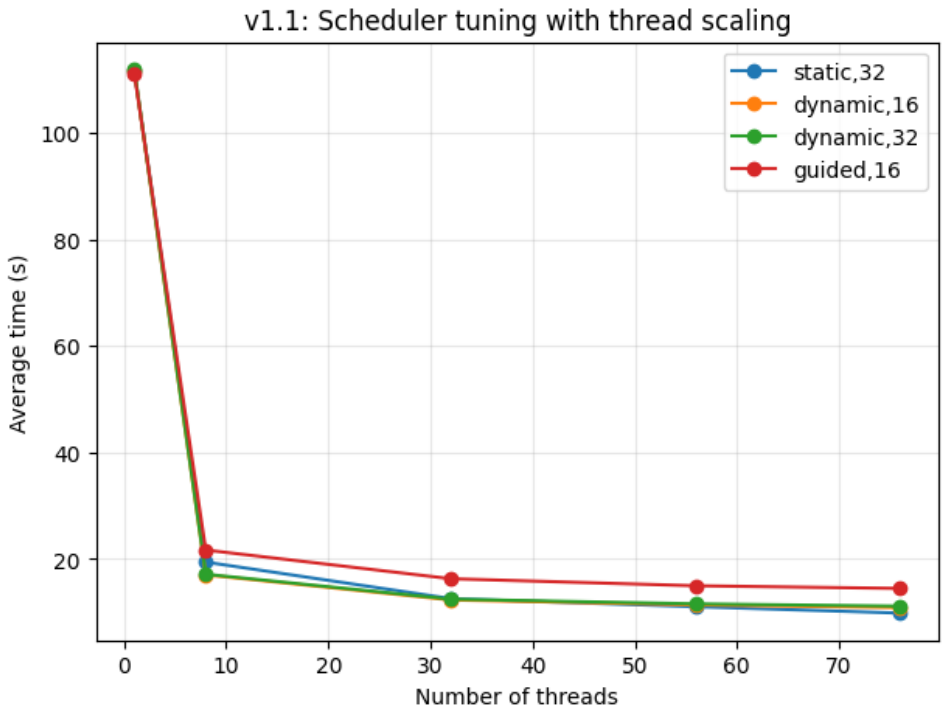


Figure 3: v1.1 scheduling tuning based on average time (s) at N=10,000, Ice Lake partition (76 threads available per node), -O3 -xHost compiler flag across different number of threads

(static,32) configuration achieved the best performance at 76 threads, with an average execution time of 9.88s. Hence, these were selected as the optimal schedule and thread count for the matrix dimension scaling experiment.

Unintuitively, the dynamic and guided schedulers did not achieve as good performance, since I originally expected them to perform better due to the varied workloads. This may partially because of the scheduling overheads. A limitation is that OMP_SCHEDULE is a single environment variable applied globally to all schedule(runtime) clauses. Independent tuning would require hardcoded schedule clauses, and hence was not pursued due to time constraints. Prioritising tuning for the dominant submatrix update may achieve better performance.

4.4 Matrix dimension scaling based on the optimal threads' usage and scheduling configuration

n	v0.4 Avg (s)	v0.4 Std (s)	v1.1 Avg (s)	v1.1 Std (s)
100	0.000039	0.000002	0.046608	0.013553
500	0.004924	0.000027	0.204882	0.002198
1000	0.056283	0.000287	0.371845	0.038283
2000	0.450845	0.001358	0.512459	0.057960
3000	1.618427	0.036542	0.601800	0.016703
4000	5.650543	0.149379	0.526178	0.317555
5000	13.164208	0.204666	0.320682	0.007552
8000	61.548106	0.388576	2.454264	0.046874
10000	122.821552	0.548808	9.989285	0.009274
15000	414.014815	1.533023	58.706428	0.154474

Table 4: Performance comparison in average and standard deviation of execution time (s) for the best single-threaded implementation (v0.4) and the optimal threads number and scheduler configuration of v1.1 (76 threads, static,32), benchmarked across matrix dimensions. (Ice Lake partition, -O3 -xHost compiler flag).

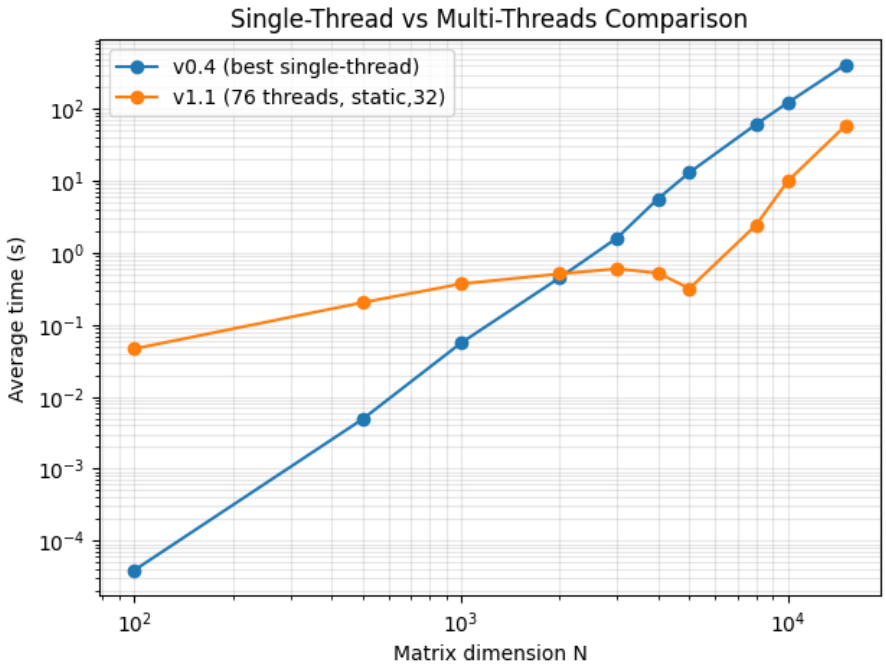


Figure 4: Performance scaling comparison against different matrix dimensions for the best single-threaded implementation (v0.4) and the optimal threads number and scheduler configuration of v1.1 (76 threads, static,32) (Ice Lake partition, -O3 -xHost compiler flag). Both axes are in log-scale for better visualisation.

As shown above, multi-threaded optimisation (v1.1) shows benefits in execution time when performing Cholesky factorisation with larger matrices, e.g., starting at around N=2000-3000, compared to the best single-threaded optimisation (v0.4) implemented. This is expected since the overheads will outweigh the multi-threads computational

benefits at smaller matrix sizes. A dip in execution time for v1.1 at N=4000-5000 is observed, which requires further investigation, and may be due to measurement variability.

5. Cache-Blocked Multi-Threaded Parallelisation (v1.2)

All performance results are obtained via compilation with -O3 -xHost compiler flag on Ice Lake partition.

5.1 v1.2: Implementations

The algorithm was restructured into a three-step process to maximise the effect of cache blocking, with block size B=64. Block size is chosen to fit into the L1 data cache size of the CPUs on Ice Lake (48KB).

At each block step:

- **Step 1**: Factorise the small BxB diagonal block (upper triangle only).
- **Step 2**: Update the panel to the right of the diagonal block. Here, the outer loop iterates over columns j and hence is parallelised with schedule(static) due to the uniform workload.
- **Step 3**: Update the trailing submatrix, with the outer loop iterates over rows i with schedule(runtime) for potential tuning. However, the schedule configuration (static, 32) was re-used from v1.1 in this coursework due to limited time. Ideally, both schedule and block size should be optimised similarly in v1.1.

Final multi-thread optimisation for the final copy is still implemented, with schedule(runtime). SIMD pragma remains implemented for vectorisation.

5.2: Performance Results

N	v0.4 (1t) Avg	v0.4 Std	v1.2 (1t) Avg	v1.2 Std	v1.1 (76t) Avg	v1.1 Std	v1.2 (76t) Avg	v1.2 Std
100	0.000039	0.000002	0.001726	0.003681	0.046608	0.013553	0.008960	0.017476
500	0.004924	0.000027	0.005466	0.000122	0.204882	0.002198	0.008700	0.000228
1000	0.056283	0.000287	0.037744	0.000446	0.371845	0.038283	0.020722	0.000614
2000	0.450845	0.001358	0.287850	0.000284	0.512459	0.057960	0.067306	0.001472
3000	1.618427	0.036542	0.971964	0.002116	0.601800	0.016703	0.099347	0.029681
4000	5.650543	0.149379	2.519240	0.003244	0.526178	0.317555	0.119923	0.001442
5000	13.164208	0.204666	5.247620	0.002509	0.320682	0.007552	0.221900	0.004511
8000	61.548106	0.388576	24.774058	0.026540	2.454264	0.046874	0.871465	0.006606
10000	122.821552	0.548808	49.277523	0.004344	9.989285	0.009274	1.642253	0.011736
15000	414.014815	1.533023	171.818312	0.092972	58.706428	0.154474	5.129327	0.013915

Table 5: Performance comparisons between v1.2 and v0.4/v1.1. (v1.2 is compiled with -O3 -xHost flag , with the following configurations “static,32”, B=64, threads=1 or 76, written in the slurm_script_test.sh, Ice Lake partition)

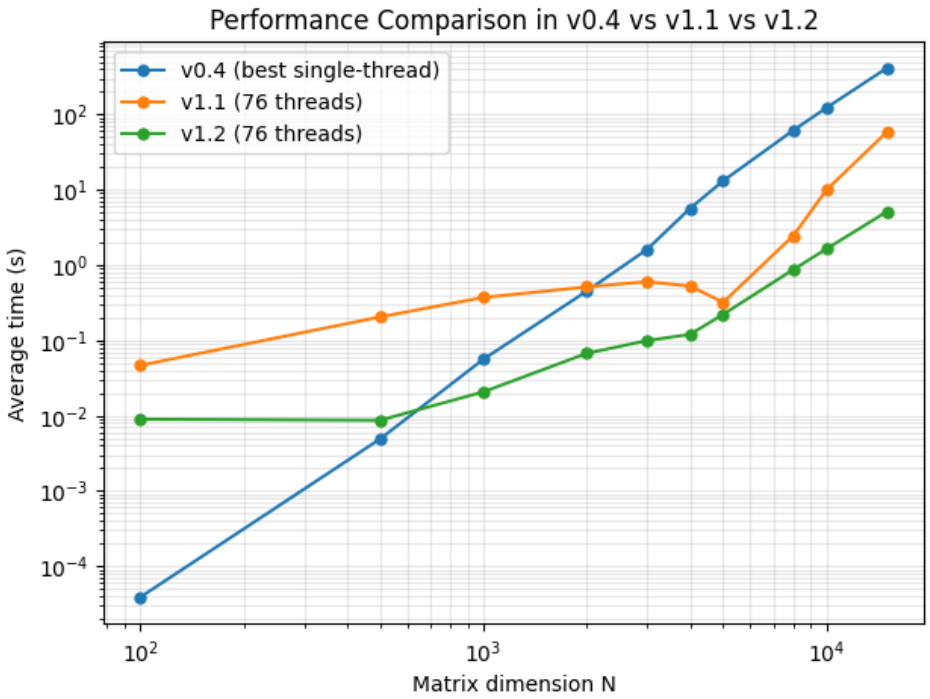


Figure 5: Performance comparison across versions v0.4 (best single-thread), v1.1 (76 threads), and v1.2 (76 threads) (Ice Lake partition, -O3 -xHost compiler flag).

At N=15000, v1.2 with 76 threads achieves a substantial reduction in execution time over v1.1 and v0.4. Even at single-thread settings, v1.2 is still much faster than v0.4 at N=15000, highlighting the contribution due to cache blocking technique to reduce cache misses.

5.3: Thread Scaling

n	1t Avg	1t Std	8t Avg	8t Std	32t Avg	32t Std	56t Avg	56t Std	76t Avg	76t Std
100	0.001726	0.003681	0.001924	0.003596	0.001572	0.002739	0.002296	0.002722	0.008960	0.017476
500	0.005466	0.000122	0.002243	0.000133	0.002633	0.000140	0.005493	0.001203	0.008700	0.000228
1000	0.037744	0.000446	0.008322	0.000196	0.006481	0.000668	0.009474	0.000456	0.020722	0.000614
2000	0.287850	0.000284	0.048054	0.000431	0.024049	0.000919	0.043137	0.000645	0.067306	0.001472
3000	0.971964	0.002116	0.165191	0.005076	0.066345	0.001315	0.063319	0.006166	0.099347	0.029681
4000	2.519240	0.003244	0.396399	0.000362	0.151463	0.001505	0.122892	0.001629	0.119923	0.001442
5000	5.247620	0.002509	0.779012	0.000115	0.286359	0.003718	0.231407	0.002459	0.221900	0.004511
8000	24.774058	0.026540	3.437327	0.005500	1.252211	0.017176	0.953727	0.020289	0.871465	0.006606
10000	49.277523	0.004344	6.684864	0.009080	2.470729	0.002539	1.894708	0.050200	1.642253	0.011736
15000	171.818312	0.092972	22.777048	0.023719	8.300334	0.003364	6.317911	0.027069	5.129327	0.013915

Table 6: v1.2 thread scaling performance with thread counts=1, 8, 32, 56, 76. Compiled with -O3 -xHost, static,32, B=64, Ice Lake partition.

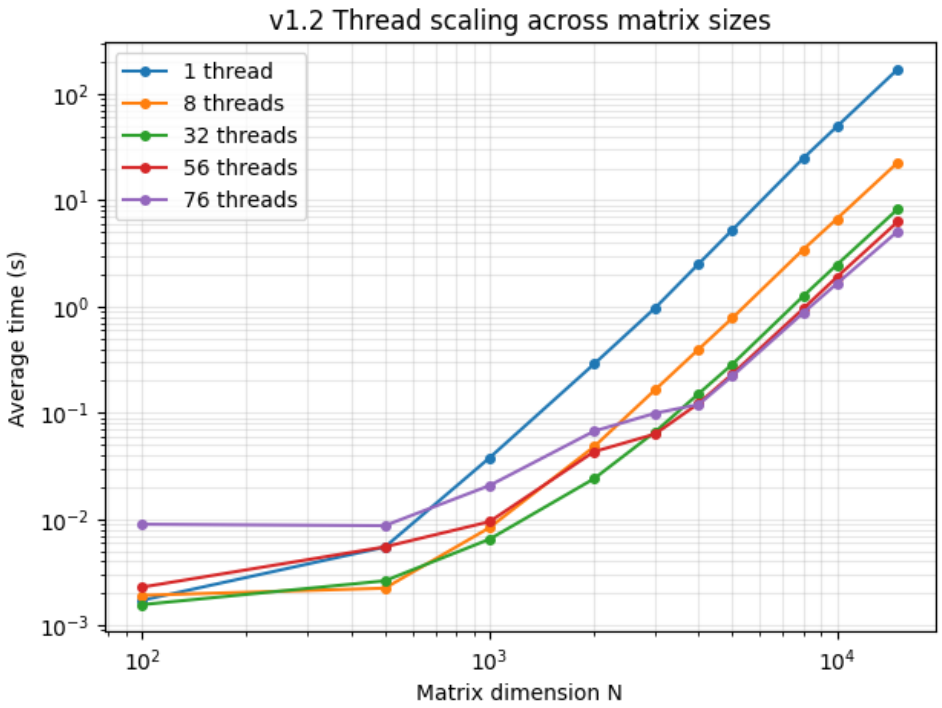


Figure 6: v1.2 thread scaling across matrix sizes (Compiled with -O3 -xHost, static,32, B=64, Ice Lake partition)

The benefits of multi-threading depends on matrix sizes, trade-off between overheads and computational benefits.

6. Practical limits on matrix sizes

The coursework specifies matrix dimension could go up to 100,000. The dimension check is implemented as requested, but stress testing at this scale was not implemented due to coursework time constraints.

7. Tests

A test suite has been developed to evaluate whether the algorithm implemented is correct (e.g., checking both output matrix and log-determinant) and whether it will correctly flag errors when needed (e.g., invalid matrix inputs under single- and multi-threaded settings). The tests are executed before every benchmark run (via Slurm script), and failed test(s) will terminate the job before benchmark begins. All implemented versions passed the tests.

8. Version Summary

Tag	Optimisation type	Key implementations
v0.0	Baseline	Baseline implementation (tested with compiler flags and partitions)
v0.1	Single thread optimisation	Loop reorder for contiguous access + hoisting
v0.2	Single thread optimisation	Reciprocal multiplication
v0.3	Single thread optimisation	Symmetry
v0.4	Single thread optimisation	SIMD + branch prediction
v1.1	Multi-threads optimisation	OpenMP tuning and optimal runs
v1.2	Multi-threads optimisation	Cache blocking

Table 7: Git tag summary. All benchmarks compiled with icpx, -O3 -xHost compiler flag on Ice Lake partition, except for baseline experiments

9. Conclusion

Overall, single-thread optimisations (v0.1-v0.4) showed good execution time improvement compared to the baseline implementation, with the greatest gain coming from loop re-ordering for contiguous access and hoisting, followed by leveraging the symmetry nature of the algorithm. Some implementations only showed moderate improvements, may attribute to the automatic optimisation handled by compiler flags.

OpenMP multi-threads optimisation (v1.1) provides further improvement in execution time for computing larger matrices (e.g., starting from n=2000-3000), but introduces overheads which outweigh the computational benefits when computing smaller matrices.

Cache blocking (v1.2) showed the strongest overall improvement compare to both the multi-threaded v1.1 version and single threaded v0.4 version, at the corresponding thread counts, confirming the benefits of reducing cache misses, with and without parallelisation.

For using the routine in other HPC systems, corresponding compiler flag should be adapted, and the optimal scheduling configuration, block size, threads number should be tuned based on the corresponding hardware.